GALACTIC PRIME TIME

Architecture Reference

Lights. Camera. Action.

v1.0 // Solo Developer Reference // Read before writing any code

Engine	Godot 4 — GDScript primary language
Platform	PC (Windows / macOS / Linux)
Database	SQLite (static game data) + JSON files (dynamic save state)
Pattern	MVC — headless simulation Model, thin Controller, Godot scene View
Network	None. Fully offline. Zero external dependencies at runtime.
Jira	galacticprimetime.atlassian.net — project KAN
Epics	KAN-1 Data Foundation → KAN-7 Progression & Audience

1. Language & Engine

Primary language: GDScript. Use GDScript for all game logic, simulation, UI, and save management. Do not mix C# into the project unless a specific plugin requires it — consistency matters more than marginal performance gains.

Godot version: Godot 4.x. All scene, node, and signal conventions follow Godot 4 API. Do not use Godot 3 patterns.

SQLite access: use the godot-sqlite plugin (GDScript wrapper). All SQLite queries go through the DAL — never call the plugin directly from a scene or simulation class.

Fonts: Share Tech Mono (monospace UI, HUD, ticker) + Rajdhani (display labels, names). Both are bundled in /assets/fonts/.

2. Folder Structure

The folder structure enforces the MVC boundary. Simulation code must never import from /scenes/ or /ui/. Scene code must never import from /simulation/ directly — all communication goes through /controller/.

```
res://

■■■ simulation/ # MODEL — pure GDScript, zero Godot node deps

■ ■■■ clock.gd # Clock event queue (priority queue by Moment)

■ ■■■ combatant.gd # CombatantState — HP, conditions, skills, inventory

■ ■■■ action_resolver.gd

■ ■■■ condition_engine.gd

■ ■■■ forced_action.gd

■ ■■■ exposure_engine.gd

■ ■■■ resistance.gd

■

■■■ controller/ # CONTROLLER — Godot-aware, owns simulation instance

■ ■■■ game_controller.gd # Autoload singleton

■ ■■■ save_manager.gd # JSON read/write for run + persistent state

■ ■■■ dal.gd # Data access layer — all SQLite queries here

■ ■■■ patron_manager.gd

■ ■■■ tag_manager.gd

■

■■■ scenes/ # VIEW — Godot scenes, no game logic

■ ■■■ combat/

■ ■ ■■■ combat_scene.tscn

■ ■ ■■■ hex_grid.tscn

■ ■ ■■■ combatant_sprite.tscn

■ ■■■ exploration/

■ ■ ■■■ exploration_scene.tscn

■ ■ ■■■ floor_map.tscn

■ ■■■ lounge/
```

```
■ ■■■ lounge_scene.tscn
■
■■■ ui/ # HUD and popup scenes
■ ■■■ hud/
■ ■ ■■■ hud.tscn # Broadcast frame + ticker
■ ■ ■■■ clock_bar.tscn
■ ■ ■■■ skill_bar.tscn
■ ■■■ popups/
■ ■■■ popup_base.tscn # Reusable popup shell
■ ■■■ party_popup.tscn
■ ■■■ character_popup.tscn
■ ■■■ goals_popup.tscn
■ ■■■ inventory_popup.tscn
■ ■■■ lounge_popup.tscn
■ ■■■ audience_popup.tscn
■
■■■ data/ # Static seed files (source of truth for SQLite)
■ ■■■ races.json
■ ■■■ skills.json
■ ■■■ conditions.json
■ ■■■ items.json
■ ■■■ modifiers.json
■ ■■■ enemies.json
■ ■■■ tags.json
■
■■■ content/ # All narrative/design content files
■ ■■■ rooms/ # One JSON per room
■ ■■■ encounters/ # One JSON per encounter
■ ■■■ dialogue/ # One JSON per dialogue tree
```

```
■ ■■■ broadcasts/ # One JSON per broadcast segment
■
■■■ assets/
■ ■■■ fonts/
■ ■■■ audio/
■ ■■■ sprites/
■
■■■ saves/ # Runtime only — created at launch, never in repo
■■■ run_state.json
■■■ checkpoint.json
■■■ persistent.json
```

3. MVC Rules — The Hard Boundaries

These rules are non-negotiable. Breaking them creates untestable, unserialisable code.

Model (simulation/)

- No imports from scenes/, ui/, or any Godot Node class.
- No preload() or load() of scene files.
- No emit_signal() to Godot scene nodes.
- All classes extend RefCounted, not Node.
- Must be fully testable headless (no display, no window).
- Every class must be fully serializable to a Dictionary for checkpoint snapshots.

Controller (controller/)

- GameController is an Autoload singleton — one instance for the entire session.
- Owns the simulation instance. Scenes never hold a reference to simulation objects.
- Translates player input events (from scenes) into simulation method calls.
- Receives simulation return values and emits Godot signals for scenes to consume.
- DAL (dal.gd) is the only file that touches SQLite. No other file imports the SQLite plugin.
- SaveManager is the only file that reads or writes JSON save files.

View (scenes/ + ui/)

- Scenes are pure presentation. They render state, they do not hold state.
- Scenes connect to GameController signals and call GameController methods.
- Scenes never call simulation methods directly.
- Scenes never read save files directly.
- All layout follows Style D: gold broadcast frame top, system UI inner panels, scrolling ticker bottom.

4. Data Layer

SQLite — Static Game Data

Contains all read-only data seeded at first launch. Never written to at runtime.

Tables: races, skills, skill_thresholds, character_skills, tags, items, modifiers, item_modifiers, conditions, condition_tiers, enemies, enemy_phases, patron_goals.

Seeding: on first launch, dal.gd checks if the DB is empty and runs the seeder, loading all /data/*.json files into SQLite. Seeder is idempotent — safe to re-run.

Access: all queries go through dal.gd. The interface exposes named methods only — get_skill(id), get_skills_by_stat(stat), get_item(id), get_race(id), get_condition(id), get_enemy(id), get_tags_for_character(character_id). No raw SQL outside dal.gd.

JSON Save Files — Dynamic State

Three files, three distinct scopes:

run_state.json	Active floor, current room, party state (HP, conditions, skills, inventory), Lounge upgrades, Exposure counts, active goals/directives, NPC relationship states. Written on every checkpoint. Restored on death.
checkpoint.json	Full snapshot of run_state.json at the last checkpoint room. This is what death restores. Overwritten each time a checkpoint room is entered.
persistent.json	Unlocked races, Patron pool (Ascended + Winner characters with Tag history and budgets), NG+ flags. Never wiped. Survives death and new runs.

Content Files — Narrative Data

All story content lives as JSON in /content/. The engine loads these at runtime — they are never baked into code. Schemas are defined in KAN-43 (rooms), KAN-44 (encounters), KAN-45 (dialogue/events). Writing content = filling in these schemas.

5. Combat Simulation

The combat simulation is a headless priority queue. It has no knowledge of Godot, scenes, or rendering. All visual feedback is driven by events emitted upward to the controller.

Clock (clock.gd)

A priority queue of CombatantAction entries sorted by Moment value (descending — Moment 10 first). Supports: schedule_action(combatant, action, moment), process_moment(), clock_reset(), add_combatant(), remove_combatant(). Returns a list of resolved events per process call.

CombatantState (combatant.gd)

Tracks per-body-part HP (dict keyed by part name), active conditions (dict keyed by condition id, value is tier), Exposed flag, Shock tier, current Moment, equipped items, active skills with levels. Must implement to_dict() and from_dict() for serialization.

Action Resolution (action_resolver.gd)

Given a CombatantState and an action definition (from DAL), checks all stat requirements. If met: auto-succeed, apply effects, schedule next moment. If not met: trigger Forced Action of appropriate type (Body or Tool), resolve action first, apply Forced Action consequences after.

Condition Engine (condition_engine.gd)

On each Clock reset, advances all active conditions by 1 tier unless Delayed. Handles all types: Bleeding, Crushed, Suffocation, Chilled, Exhausted, Infected, Burn, Poison (with spread rules), Dissolution. Applies Shock at specified tier. Returns list of advancement events.

Key Combat Rules (enforce in code)

- Actions auto-succeed when stat requirements met — NO to-hit rolls.
- Multi-Moment actions: character is Exposed for duration, cannot act until resolved.
- Forced Action consequences apply AFTER action resolution, not before.
- Head can only be targeted when Exposed, Helpless, or Overwhelmed.
- Death: Head HP = 0 or Torso HP = 0. Bleed-out state if condition active on death.
- Surface immunity (boss-specific): damage blocked until breach condition met.

6. Save & Checkpoint System

On entering a checkpoint room: SaveManager serializes full CombatantState for all party members + run state into checkpoint.json and run_state.json simultaneously.

On death: SaveManager loads checkpoint.json, reinitializes all CombatantStates from the snapshot, resets room state to checkpoint room. Lounge upgrades and persistent.json are untouched.

On Ascension: the Ascending character's final state (Exposure counts, Tag history) is written into persistent.json as a Patron entry. They are removed from run_state party roster. Character creation flow is triggered.

On win: winning character's race(s) are appended to persistent.json unlocked_races array. Character is added to persistent.json patron_pool as a Winner-type Patron.

Serialization Contract

Every simulation class must implement:

```
func to_dict() -> Dictionary:

return {

"class": "CombatantState",

"hp": body_parts,

"conditions": active_conditions,

"skills": skill_levels,

# ... all state

}


static func from_dict(data: Dictionary) -> CombatantState:

var c = CombatantState.new()

c.body_parts = data["hp"]

# ... restore all state

return c
```

7. Signal & Event Convention

GameController defines all signals. Scenes connect to GameController signals — they never connect to each other directly.

Combat signals (emitted by GameController after processing simulation events):

```
signal action_resolved(combatant_id, action_id, targets, effects)

signal condition_advanced(combatant_id, condition_id, new_tier)

signal combatant_died(combatant_id)

signal clock_moment_changed(current_moment)

signal clock_reset()

signal forced_action_triggered(combatant_id, type, roll_result)

signal exposed_state_changed(combatant_id, is_exposed)
```

Exploration signals:

```
signal room_entered(room_id, room_data)

signal encounter_triggered(encounter_id)

signal npc_reputation_changed(npc_id, new_score, delta)

signal checkpoint_reached(room_id)

signal dialogue_started(dialogue_id)

signal broadcast_triggered(broadcast_id)
```

Exposure signals:

```
signal viewers_changed(character_id, new_count, delta)

signal new_patron_earned(character_id, patron_token_awarded)

signal goal_completed(goal_id, reward)

signal tag_acquired(character_id, tag_id)

signal tag_fading(character_id, tag_id)
```

8. Naming Conventions

Files	snake_case.gd / snake_case.tscn — always
Classes	PascalCase — class_name CombatantState
Variables	snake_case — var current_moment: int
Constants	SCREAMING_SNAKE — const MAX_MOMENT = 10
Signals	snake_case past tense — signal action_resolved(...)
Functions	snake_case verb — func process_moment() -> Array
IDs	Always int in SQLite. Always string in JSON (use str(id) when crossing boundary).
Body parts	String keys: "head", "torso", "left_arm", "right_arm", "left_leg", "right_leg"
Conditions	String keys matching SQLite id field — "bleeding", "crushed", "suffocation" etc.
Save keys	snake_case strings matching class field names exactly — no aliases

Type annotations

All function signatures must be fully typed. GDScript 4 enforces this at export — do it from the start.

```
# CORRECT

func get_skill(skill_id: int) -> Dictionary:

func schedule_action(combatant: CombatantState, action_id: int, moment: int) -> void:


# WRONG — no types

func get_skill(id):

func schedule_action(c, a, m):
```

9. NPC & Narrative Systems

NPC Relationship Model

Static NPC definitions live in SQLite: id, name, floor, role, base_inclinations (array of {tag_id, modifier} — positive or negative). These never change at runtime.

Dynamic NPC state lives in run_state.json under npc_relationships: {npc_id: {reputation: int, known_tags: [], dialogue_state: string}}. Reputation is a numeric score. Tags act as multipliers on reputation gain/loss. Tags also unlock dialogue branches regardless of score.

Event / Dialogue System

Events are loaded from /content/dialogue/*.json and /content/broadcasts/*.json. The event system in GameController processes event triggers (room_entry, combat_trigger, broadcast_segment, lore_reveal) and fires the appropriate Godot signal. Dialogue trees are node graphs — each node has text, conditions, and choices with next_node_id pointers.

Quest State Machine

Quests and directives are stored in run_state.json as {quest_id: {status: "available|active|completed|failed", trigger_conditions: []}}. The GameController evaluates trigger conditions against current world state at key moments: room entry, combat end, NPC interaction, clock reset.

10. UI Conventions

All UI follows Style D: hybrid broadcast + system UI. The outer broadcast frame (top bar + bottom ticker) is always visible. Inner panels are dark system UI cards. See mockup_combat.html and mockup_exploration.html for reference.

Colour palette (CSS → Godot Color):

Background	#0c0c10 — main screen background
Panel	#14141c — system UI card background
Panel raised	#1c1c28 — elevated card / button background
Border	#2a2a40 — all panel borders, 0.5px
Gold	#d4a820 — primary accent, broadcast frame, active elements
Red	#cc2233 — enemy, danger, LIVE badge, critical conditions
Green	#3a6a3a — party hex fill
Dim text	#6666aa — labels, system text, ticker
Body text	#e0e0e8 — standard readable text
Code text	#c8e8a0 — monospace values in HUD

HUD layout structure:

- Top bar: broadcast frame (show title, episode, viewer count, LIVE badge). In combat: Clock bar replaces left section.
- Bottom bar: scrolling ticker with live session info.
- Left panel: party status (HP per body part, conditions).
- Right panel: audience (viewers/followers/patrons), active goal, selected target.
- Centre: hex arena (combat) or floor map (exploration).
- Popups: all use popup_base.tscn — draggable, stackable, close button.

11. Development Order

Follow epic order in Jira. Do not start a later epic until the prior epic's core tasks are done. The simulation must be headless-testable before any scene is built.

KAN-1	Data Foundation — SQLite schema, seed files, DAL, JSON save schemas. Nothing else works without this.
KAN-2	Core Combat Engine — Clock, CombatantState, action resolver, condition engine, forced action, resistance. All headless. Write unit tests.
KAN-3	Godot Scaffolding — project setup, folder structure, GameController singleton, hex grid renderer, SaveManager.
KAN-4	Character & Party — character creation, localized HP, skill runtime, inventory, party deployment & bench.
KAN-5	Map & Exploration — room state machine, Floor 1 map data, exploration controller, checkpoint & rewind.
KAN-6	UI Layer — base HUD, popup framework, all popup content scenes, combat HUD elements.
KAN-7	Progression & Audience — Exposure engine, Tag runtime, Goal & Directive system, Ascension, NG+ persistent state.

Content (parallel track after KAN-3):

Room content, encounter files, and dialogue trees can be written in JSON at any time after the schemas are defined (KAN-43/44/45). Content writing does not depend on engine implementation being complete.

Boss encounter (Tutorial — Incineradile):

The tutorial boss is fully designed in TTRPG format. When implementing in KAN-2/KAN-3: the boss is an Enemy with 6 phases, surface immunity (breach_condition required), a 50HP mycelium network as its actual health pool, a destroyable 30HP flamethrower arm, death explosion with 19-space instant-kill radius, and a wall-bouncing dash that scales in complexity per phase. All of this maps to the encounter JSON schema defined in KAN-44.

12. What Not To Do

Hard rules. These exist because they've already been thought through.

- Do not add MongoDB. The decision was made: SQLite for static, JSON for dynamic. Done.
- Do not use PostgreSQL or any server-based database. This is an offline single-player game.
- Do not put game logic in scene scripts. Scenes are views only.
- Do not call the SQLite plugin from anywhere except dal.gd.
- Do not read or write save files from anywhere except save_manager.gd.
- Do not use to-hit rolls. Actions auto-succeed when stat requirements are met. Forced Action handles the failure path.
- Do not use localStorage or any browser API. This is a native Godot app.
- Do not add live service features, telemetry, or internet requirements.
- Do not design encounters where the win condition is just 'deal enough damage'. Bosses require discovery of a win condition — surface immunity, breach path, phase unlock, etc.
- Do not skip writing unit tests for simulation classes. The headless layer is the game's foundation — bugs there corrupt everything above it.